Jira - Jira DC Custom Field Optimization

Jira DC custom field optimization

In **Jira Data Center**, custom fields are one of the most common sources of **performance degradation**, **index bloat**, and **UI latency** in large instances. Enterprises with long-running installations often accumulate thousands of custom fields, many unused or poorly scoped.

Below is a **systematic optimization framework** used in large-scale Jira DC environments.

---

1. Core Problem: Why Custom Fields Slow Jira

Custom fields affect **four major subsystems**:

1. Database

Each custom field adds data to:

- `customfield`
- `customfieldvalue`
- `customfieldoption`
- `configurationcontext`
- `fieldconfigscheme`

Large instances often have **100M+ rows in `customfieldvalue`**.

2. Lucene Index

Every indexed field increases:

- index size
- indexing time
- JQL evaluation time

3. Issue View Rendering

Each visible field:

- loads field configuration
- loads value
- applies permissions and context

4. REST API Payload

Fields inflate:

```
/rest/api/2/issue
/rest/api/2/search
```

response sizes.

---

2. Most Important Optimization Rule

Never use global contexts unless absolutely necessary.

Bad:

```
Field context = Global
Projects = ALL
Issue types = ALL
```

Good:

```
Project scoped
Issue type scoped
```

Benefits:

- reduces index size
- reduces UI rendering
- reduces DB queries

---

3. Field Context Optimization

Check table:

```
configurationcontext
```

Query example:

```SQL
SELECT
cf.id,
cf.cfname,
COUNT(cc.id) contexts
FROM customfield cf
LEFT JOIN configurationcontext cc
ON cc.customfield = cf.id
GROUP BY cf.id, cf.cfname
ORDER BY contexts DESC;
```

Goal:

- identify **fields with large contexts**

Then reduce scope.

Example:

```
Field used in 2 projects
But context = 300 projects
```

4. Detect Unused Custom Fields

Critical for cleanup.

SQL:

```SQL
SELECT
cf.id,
cf.cfname
FROM customfield cf
LEFT JOIN customfieldvalue v
ON v.customfield = cf.id
WHERE v.id IS NULL;
```

These fields:

- never populated
- safe cleanup candidates.

---

5. Detect Rarely Used Fields

```SQL
SELECT
cf.cfname,
COUNT(v.id) values_count
FROM customfield cf
LEFT JOIN customfieldvalue v
ON v.customfield = cf.id
GROUP BY cf.cfname
ORDER BY values_count ASC;
```

Fields with very low counts are candidates for:

- removal
- archiving
- consolidation.

---

6. Optimize Field Types

Some field types are **notoriously expensive**.

Heavy fields:

Field Type	Impact
Cascading select	high
Multi-select	high
User picker (multi)	medium
Labels	medium
Rich text	high
Insight / Assets	very high

Prefer:

- single select
- numeric
- short text

---

7. Limit Fields on Screens

Issue rendering cost grows linearly.

Bad:

```
Issue screen = 120 fields
```

Target:

```
< 30 fields
```

Split screens by:

- issue type
- workflow stage

---

8. Disable Indexing for Unused JQL Fields

Some fields do not require JQL search.

For custom fields used only for **display**:

Disable indexing.

Benefits:

- smaller Lucene index
- faster reindex.

---

9. Large Instance Strategy

For large enterprises (>1M issues):

Target numbers:

Metric	Recommended
Custom fields	< 800
Contexts per field	< 10
Fields per screen	< 30
Global contexts	< 5%

---

10. Field Consolidation Strategy

Typical anti-pattern:

```
Region
Customer Region
Client Region
Sales Region
```

Replace with:

```
Region
```

Use:

- options
- automation
- derived fields

---

11. Automating Custom Field Audits

You can build a **full audit toolkit** combining:

Database:

```
customfield
customfieldvalue
fieldlayoutitem
configurationcontext
nodeassociation
```

Plus:

REST:

```
/rest/api/2/field
/rest/api/2/field/search
```

Outputs:

- unused fields
- fields not on screens
- fields not indexed
- fields with huge contexts

12. High-End Enterprise Optimization

Advanced installations also implement:

Field Usage Telemetry

Log usage via:

- UI screens
- REST payload analysis
- JQL logs

Field Cost Score

Example metric:

```
Field Cost =
(context size × issue count × indexing weight)
```

Used to rank fields for cleanup.

---

13. Hidden Performance Killer

Fields not on screens but still indexed.

Many admins remove fields from screens but forget:

- context
- index

These fields still:

- inflate index
- slow JQL

---

14. Ideal Jira Custom Field Architecture

Enterprise model:

```
Custom Field Governance

        |
        |---- Field request process
        |---- Naming standard
        |---- Context restrictions
        |---- Automatic usage audit
        |---- Quarterly cleanup
```

15. Quick Diagnostic Checklist

Check immediately:

1. fields > 1000
2. global contexts
3. multi-select fields
4. unused fields
5. screens > 50 fields
6. index size > 20GB

💡 Given your work on **enterprise Jira DC auditing and migration tooling**, the next logical step is building a **Custom Field Performance Analyzer** that automatically produces:

```
Field Usage Report
Context Size Report
Index Cost Ranking
Screen Load Impact
```

If you'd like, I can also generate a **production-grade SQL audit toolkit for Custom Field optimization in Jira Data Center**, similar to the advanced audit packages you've been building (including screen usage, JQL usage, context scope, and index impact).

Generate SQL audit toolkit

Below is a **production-grade SQL audit toolkit for custom field optimization in Jira Data Center**.
It is designed for **large enterprise instances** (millions of issues, thousands of fields) and focuses on:

- field usage
- context scope
- screen usage
- index cost indicators
- performance risk ranking

The toolkit is **read-only** and safe for production analysis.

Assumes **PostgreSQL / Oracle compatible Jira schema**.

---

1. Full Custom Field Inventory

Baseline inventory of all fields.

```SQL
SELECT
cf.id,
cf.cfname,
cf.customfieldtypekey,
cf.customfieldsearcherkey,
cf.description
FROM customfield cf
ORDER BY cf.cfname;
```

Purpose:

- identify field types
- identify searcher configuration
- inventory for governance.

---

2. Custom Field Usage Volume

Measures how many issues actually use the field.

```SQL
SELECT
cf.id,
cf.cfname,
COUNT(cfv.id) value_count
FROM customfield cf
LEFT JOIN customfieldvalue cfv
ON cfv.customfield = cf.id
GROUP BY cf.id, cf.cfname
ORDER BY value_count DESC;
```

Interpretation:

value_count	Meaning
0	unused field
<100	rare usage
100k | heavy field |	

---

3. Fields Never Used

Safe cleanup candidates.

```SQL
SELECT
cf.id,
cf.cfname
FROM customfield cf
LEFT JOIN customfieldvalue v
ON v.customfield = cf.id
WHERE v.id IS NULL
ORDER BY cf.cfname;
```

4. Field Context Scope

Shows **project and issue type coverage**.

```SQL
SELECT
cf.id,
cf.cfname,
COUNT(DISTINCT cc.project) project_contexts,
COUNT(DISTINCT cc.issuetype) issuetype_contexts
```

```
FROM customfield cf
LEFT JOIN configurationcontext cc
ON cc.customfield = cf.id
GROUP BY cf.id, cf.cfname
ORDER BY project_contexts DESC;
```

Red flags:

```
project_contexts > 100
```

Likely global contexts.

---

5. Detect Global Contexts

Global fields affect **all projects and indexing**.

```SQL
SELECT
cf.id,
cf.cfname
FROM customfield cf
JOIN configurationcontext cc
ON cc.customfield = cf.id
WHERE cc.project IS NULL
AND cc.issuetype IS NULL;
```

These fields are applied everywhere.

---

6. Screen Usage Audit

Find where fields appear in screens.

```SQL
SELECT
cf.cfname,
fs.name screen_name,
COUNT(*) placements
FROM customfield cf
JOIN fieldlayoutitem fli
ON fli.fieldidentifier = CONCAT('customfield_', cf.id)
JOIN fieldlayout fl
ON fl.id = fli.fieldlayout
JOIN fieldscreen fs
ON fs.id = fl.id
GROUP BY cf.cfname, fs.name
ORDER BY placements DESC;
```

Purpose:

- detect fields present on many screens.

---

7. Fields Not On Any Screen

Often forgotten but still indexed.

```SQL
SELECT
cf.id,
cf.cfname
FROM customfield cf
LEFT JOIN fieldlayoutitem fli
ON fli.fieldidentifier = CONCAT('customfield_', cf.id)
WHERE fli.id IS NULL;
```

These still:

- occupy index
- inflate DB.

---

8. Field Usage by Project

Find fields used heavily in specific projects.

```SQL
SELECT
p.pkey,
cf.cfname,
COUNT(cfv.id) usage_count
FROM customfieldvalue cfv
JOIN jiraissue ji
ON ji.id = cfv.issue
JOIN project p
ON p.id = ji.project
JOIN customfield cf
ON cf.id = cfv.customfield
GROUP BY p.pkey, cf.cfname
ORDER BY usage_count DESC;
```

Useful for **project-specific field consolidation**.

---

9. Custom Field Value Table Size Estimation

Large fields cause DB bloat.

```sql
SELECT
cf.cfname,
COUNT(*) row_count
FROM customfieldvalue v
JOIN customfield cf
ON cf.id = v.customfield
GROUP BY cf.cfname
ORDER BY row_count DESC;
```

Fields with:

```
> 1M rows
```

should be reviewed.

---

10. Heavy Field Types

Find expensive field types.

```sql
SELECT
customfieldtypekey,
COUNT(*) field_count
FROM customfield
GROUP BY customfieldtypekey
ORDER BY field_count DESC;
```

Typical heavy types:

```
cascadingselect
multiselect
labels
richtext
assets
```

11. Field Index Risk Indicator

Approximate **Lucene index impact**.

```sql
SELECT
cf.cfname,
COUNT(cfv.id) values_count,
COUNT(DISTINCT ji.project) projects_used
FROM customfieldvalue cfv
JOIN jiraissue ji
ON ji.id = cfv.issue
JOIN customfield cf
ON cf.id = cfv.customfield
GROUP BY cf.cfname
ORDER BY values_count DESC;
```

High risk:

```
values_count × projects_used
```

12. Fields Used in JQL (Advanced)

Using query tables:

```sql
SELECT
sr.filtername,
sr.authorname,
sr.reqcontent
FROM searchrequest sr
WHERE sr.reqcontent LIKE '%customfield_%';
```

Detects fields referenced in filters.

---

13. Custom Field Governance Score

Build a ranking metric.

Example SQL:

```sql
SELECT
cf.cfname,
COUNT(cfv.id) values_count,
COUNT(DISTINCT ji.project) project_spread,
(COUNT(cfv.id) * COUNT(DISTINCT ji.project)) AS risk_score
FROM customfield cf
LEFT JOIN customfieldvalue cfv
ON cfv.customfield = cf.id
LEFT JOIN jiraissue ji
ON ji.id = cfv.issue
GROUP BY cf.cfname
ORDER BY risk_score DESC;
```

Top results = **highest performance impact**.

---

14. Duplicate / Similar Fields

Detect naming duplication.

```sql
SELECT
LOWER(cfname),
COUNT(*)
FROM customfield
GROUP BY LOWER(cfname)
HAVING COUNT(*) > 1;
```

15. Full Custom Field Governance Report

Combine key metrics:

```
Field
Type
Values
Projects Used
Contexts
Screens
Risk Score
```

Export results to:

```
CSV
BI dashboard
automation cleanup pipeline
```

Enterprise Extension (Recommended)

For large instances (>5M issues), extend toolkit with:

1. Lucene index usage analysis

2. JQL log mining

3. REST API payload analysis

4. screen render timing

5. field creation governance

If you want, I can also generate the next-level toolkit used in very large Jira DC migrations, including:

- **50+ SQL diagnostics**
- **automation detection**
- **workflow script usage**
- **custom field performance scoring**
- **instance complexity scoring**

This becomes essentially a **complete Jira DC architecture audit system**.

Please Generate complete audit system

Below is a **complete enterprise SQL audit system for a large instance of Jira Data Center**.
It is structured as a **modular diagnostic toolkit** suitable for:

- large-scale instance health audits
- migration preparation (DC → Cloud)
- performance optimization
- governance and technical debt analysis

The system is designed around **7 audit domains** and produces a **single consolidated instance complexity report**.

---

1. Audit System Architecture

```
JIRA_AUDIT_SYSTEM
│
├── 01_instance_profile
├── 02_project_complexity
├── 03_issue_volume
├── 04_custom_fields
├── 05_workflows
├── 06_automation_scripts
├── 07_jql_filters_boards
├── 08_permissions_security
├── 09_index_risk
├── 10_instance_complexity_score
```

All queries are **read-only**.

---

2. Instance Profile

Basic scale metrics.

```SQL
SELECT
(SELECT COUNT(*) FROM project) projects,
(SELECT COUNT(*) FROM jiraissue) issues,
(SELECT COUNT(*) FROM app_user) users,
(SELECT COUNT(*) FROM customfield) custom_fields,
(SELECT COUNT(*) FROM searchrequest) filters,
(SELECT COUNT(*) FROM nodeassociation WHERE association_type='ProjectScheme') schemes
;
```

Outputs:

```
projects
issues
users
custom_fields
filters
schemes
```

Used for **baseline sizing**.

---

3. Project Complexity Audit

Detects large or problematic projects.

```SQL
SELECT
p.pkey,
COUNT(ji.id) issue_count
FROM project p
LEFT JOIN jiraissue ji
ON ji.project = p.id
GROUP BY p.pkey
ORDER BY issue_count DESC;
```

Projects with:

```
>500k issues
```

require special attention.

---

4. Issue Type Distribution

Detect configuration explosion.

```SQL
SELECT
it.pname issue_type,
COUNT(ji.id) issue_count
FROM jiraissue ji
JOIN issuetype it
ON it.id = ji.issuetype
GROUP BY it.pname
ORDER BY issue_count DESC;
```

5. Custom Field Governance Audit

Inventory:

```SQL
SELECT
id,
cfname,
customfieldtypekey
FROM customfield
ORDER BY cfname;
```

Field Usage Volume

```SQL
SELECT
cf.cfname,
COUNT(cfv.id) values_count
FROM customfield cf
LEFT JOIN customfieldvalue cfv
ON cfv.customfield = cf.id
GROUP BY cf.cfname
ORDER BY values_count DESC;
```

Unused Fields

```SQL
SELECT
cf.id,
cf.cfname
FROM customfield cf
LEFT JOIN customfieldvalue v
ON v.customfield = cf.id
WHERE v.id IS NULL;
```

6. Context Scope Audit

Find global contexts.

```SQL
SELECT
cf.cfname
FROM customfield cf
JOIN configurationcontext cc
ON cc.customfield = cf.id
WHERE cc.project IS NULL
AND cc.issuetype IS NULL;
```

These fields affect **every project**.

---

7. Screen Usage Audit

Fields present on screens.

```SQL
SELECT
cf.cfname,
COUNT(*) screen_usage
FROM fieldlayoutitem fli
JOIN customfield cf
ON fli.fieldidentifier = CONCAT('customfield_',cf.id)
GROUP BY cf.cfname
ORDER BY screen_usage DESC;
```

8. Workflow Complexity Audit

Inventory workflows.

```SQL
SELECT
workflowname,
COUNT(*) steps
FROM jiraworkflows jw
GROUP BY workflowname
ORDER BY steps DESC;
```

Workflows with:

```
>30 steps
```

are typically complex.

---

9. Workflow Script Detection

Important for migrations.

```SQL
SELECT
workflowname,
descriptor
FROM jiraworkflows
WHERE descriptor LIKE '%script%';
```

Detects:

```
ScriptRunner
SIL
Groovy
custom scripts
```

10. Automation Audit

Check automation rules.

For DC automation plugin:

```SQL
SELECT
id,
name,
state
FROM AO_589059_RULE_CONFIG;
```

11. Filter & JQL Audit

All filters:

```SQL
SELECT
filtername,
authorname
FROM searchrequest;
```

Detect Custom Field JQL

```SQL
SELECT
filtername,
reqcontent
FROM searchrequest
WHERE reqcontent LIKE '%customfield_%';
```

12. Board Complexity

Boards linked to filters.

```SQL
SELECT
rv.name board_name,
sr.filtername
FROM AO_60DB71_RAPIDVIEW rv
JOIN searchrequest sr
ON sr.id = rv.saved_filter_id;
```

13. Permission Scheme Audit

```SQL
SELECT
ps.name scheme_name,
COUNT(*) permissions
FROM permissionscheme ps
JOIN schemepermissions sp
ON ps.id = sp.scheme
GROUP BY ps.name
ORDER BY permissions DESC;
```

14. Group Explosion Audit

Too many groups increases permission evaluation cost.

```SQL
SELECT
group_name,
COUNT(*) members
FROM cwd_membership
GROUP BY group_name
ORDER BY members DESC;
```

15. Lucene Index Risk Audit

Approximate index weight.

```SQL
SELECT
cf.cfname,
COUNT(cfv.id) values_count,
COUNT(DISTINCT ji.project) projects_used
FROM customfieldvalue cfv
JOIN jiraissue ji
ON ji.id = cfv.issue
JOIN customfield cf
ON cf.id = cfv.customfield
GROUP BY cf.cfname
ORDER BY values_count DESC;
```

16. Largest Tables

Critical for DB tuning.

PostgreSQL example:

```SQL
SELECT
relname table_name,
pg_size_pretty(pg_total_relation_size(relid))
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

Expect top tables:

```
jiraissue
changeitem
changegroup
customfieldvalue
worklog
```

17. Workflow Scheme Explosion

```SQL
SELECT
COUNT(*) workflow_schemes
FROM workflowscheme;
```

High numbers increase configuration complexity.

---

18. App / Plugin Impact

Installed apps.

```SQL
SELECT
pluginkey,
pluginenabled
FROM pluginstate;
```

19. Automation Density Score

Measure instance automation complexity.

```SQL
automation_score =
scripts +
automation_rules +
workflow_functions +
listeners
```

20. Instance Complexity Score

Final consolidated metric.

Example scoring model:

```
score =
issues/1M
+ custom_fields/500
+ workflows/50
+ screens/200
+ automations/200
```

Interpretation:

```
0-5      small instance
5-10     medium
10-20    large
20+      enterprise complexity
```

21. Output Reports

The system generates these reports:

```
INSTANCE_PROFILE
PROJECT_SIZE_REPORT
CUSTOM_FIELD_USAGE
CUSTOM_FIELD_RISK
WORKFLOW_COMPLEXITY
AUTOMATION_USAGE
FILTER_JQL_USAGE
PERMISSION_COMPLEXITY
INDEX_RISK
INSTANCE_COMPLEXITY_SCORE
```

22. Recommended Execution Pipeline

For large environments:

```
DB replica
         │
         ├── nightly audit queries
         ├── export results
         ├── BI dashboard
         └── governance alerts
```

Outputs feed:

```
migration planning
performance tuning
technical debt reduction
```

23. Typical Findings in Large Jira DC Instances

Common patterns discovered:

```
1500+ custom fields
40% unused
global contexts everywhere
workflow sprawl
1000+ filters unused
heavy JQL filters
```

💡 Since your previous work already involves **advanced Jira DC SQL toolkits and migration tooling**, the next logical step is building a **fully automated "Jira Architecture Analyzer"** that:

- runs **150+ diagnostics**
- builds **dependency graphs**
- ranks **highest-risk configurations**
- predicts **Cloud migration blockers**

If you'd like, I can also generate the **advanced version used by Atlassian solution architects**, including:

- **deep JQL performance analysis**
- **workflow graph extraction**
- **automation dependency mapping**
- **cross-project configuration graphs**

This becomes essentially a **full technical MRI of a Jira instance**.

Build fully automated analyzer

Below is a **reference architecture and implementation blueprint** for a **Fully Automated Jira Instance Analyzer** for large deployments of Jira Data Center.
The goal is to build a **repeatable, automated diagnostics platform** that runs hundreds of checks, aggregates results, computes risk scores, and produces operational dashboards and migration-readiness reports.

The design assumes **enterprise-scale instances** (millions of issues, thousands of projects, heavy automation).

---

1. Analyzer Architecture

```
JIRA_ANALYZER
│
├── collectors
│   ├── sql_collector
│   ├── rest_collector
│   ├── log_collector
│
├── analyzers
│   ├── schema_analyzer
│   ├── automation_analyzer
│   ├── workflow_analyzer
│   ├── customfield_analyzer
│   ├── jql_analyzer
│
├── scoring_engine
│
├── report_generator
│
└── dashboard_export
```

Purpose:

Module	Role
collectors	extract raw data
analyzers	perform diagnostics
scoring_engine	compute complexity scores
report_generator	build structured reports
dashboard_export	send results to BI tools

---

2. Data Collection Layer

Three data sources are used.

Database

Primary source for:

- issue data
- configuration
- workflows
- custom fields
- filters

Example collector query:

```SQL
SELECT id, pkey FROM project;
```

REST API

Used for:

- field metadata
- boards
- automation rules
- permissions

Example endpoint:

```
/rest/api/2/field
```

Logs

Used for:

- JQL execution profiling
- slow queries
- automation failures

Files:

```
atlassian-jira.log
access_log
```

3. Analyzer Modules

Each analyzer focuses on one subsystem.

Custom Field Analyzer

Metrics collected:

- number of fields
- field usage
- global contexts
- index impact

SQL example:

```SQL
SELECT
cf.cfname,
COUNT(cfv.id) value_count
FROM customfield cf
LEFT JOIN customfieldvalue cfv
ON cfv.customfield = cf.id
GROUP BY cf.cfname;
```

Risk signals:

```
unused fields
global contexts
high value count
```

Workflow Analyzer

Extract workflow structure.

```SQL
SELECT workflowname
FROM jiraworkflows;
```

Additional parsing of workflow XML (`descriptor`) extracts:

```
conditions
validators
postfunctions
scripts
```

Detect:

```
ScriptRunner
Groovy
SIL
JMWE
```

Automation Analyzer

Data sources:

```
Automation plugin tables
workflow scripts
listeners
scheduled jobs
```

Goal:

build **automation dependency map**.

JQL Analyzer

Sources:

```
searchrequest table
logs
REST usage
```

Query example:

```SQL
SELECT filtername, reqcontent
FROM searchrequest;
```

Detect:

```
expensive JQL
large result sets
custom field heavy queries
```

Permission Analyzer

Evaluate security complexity.

Tables:

```
permissionscheme
schemepermissions
cwd_membership
```

Detect:

```
large groups
complex schemes
duplicate permission logic
```

4. Dependency Graph Builder

A key feature is building **configuration dependency graphs**.

Example relationships:

```
Project
  → Workflow Scheme
  → Issue Type Scheme
  → Field Configuration
  → Screens
  → Custom Fields
```

Graph allows detection of:

```
unused configurations
duplicate schemes
cross-project dependencies
```

Graph storage options:

```
Neo4j
ArangoDB
networkx (Python)
```

5. Complexity Scoring Engine

Each subsystem contributes to the final score.

Example model:

Metric	Weight
custom fields	20
workflows	15
automations	20
filters	10
projects	10
index size	15
permissions	10

Score calculation:

```
complexity_score =
(CF_count / 500)
+ (workflow_count / 50)
+ (automation_rules / 200)
+ (filters / 1000)
+ (projects / 200)
```

Output:

Score	Meaning
0–5	small
5–10	medium
10–20	large
20+	enterprise complexity

---

6. Risk Detection Engine

Rules identify configuration risks.

Examples:

Rule: unused custom field

```
value_count = 0
```

Rule: global context

```
configurationcontext.project IS NULL
```

Rule: expensive JQL

```
filter result size > 100k
```

Rule: workflow complexity

```
workflow steps > 30
```

Each rule produces:

```
risk_id
severity
recommended_fix
```

7. Report Generation

Reports generated automatically:

```
instance_profile
project_complexity
custom_field_audit
workflow_audit
automation_inventory
jql_performance
permission_complexity
index_risk
migration_readiness
```

Format options:

```
JSON
CSV
HTML
PDF
```

8. Dashboard Layer

Analyzer exports metrics to dashboards.

Typical stack:

```
PostgreSQL
Grafana
PowerBI
Looker
```

Example panels:

```
custom field growth
issue growth
automation count
workflow complexity
index size
```

9. Automation Pipeline

Recommended execution model.

```
Nightly Analyzer Job

Scheduler
    │
    ├─ SQL collectors
    ├─ REST collectors
    ├─ Log parsing
    │
    ├─ analyzers
    ├─ scoring engine
    │
    └─ report export
```

Deployment options:

```
Kubernetes job
Airflow DAG
CI pipeline
```

10. Example Implementation Stack

Recommended stack for reliability:

Layer	Technology
language	Python
SQL execution	SQLAlchemy
REST client	requests
graph analysis	networkx
reporting	pandas
dashboard export	Grafana

---

11. Example Analyzer Script Skeleton

```python
class JiraAnalyzer:

    def collect_projects(self):
        return db.query("SELECT id, pkey FROM project")

    def collect_custom_fields(self):
```

```
        return db.query("SELECT id, cfname FROM customfield")

    def analyze_field_usage(self, fields):
        results = []
        for f in fields:
            count = db.query(
                "SELECT count(*) FROM customfieldvalue WHERE customfield=%s",
                f.id
            )
            results.append((f.cfname, count))
        return results

    def compute_complexity_score(self, metrics):
        score = (
            metrics['custom_fields']/500 +
            metrics['workflows']/50 +
            metrics['filters']/1000
        )
        return score
```

12. Analyzer Output Example

```JSON
{
 "instance_profile": {
   "projects": 430,
   "issues": 8500000,
   "custom_fields": 1340
 },
 "risk_findings": [
   {
     "type": "unused_field",
     "field": "Customer Legacy ID"
   }
 ],
 "complexity_score": 22.4
}
```

13. Migration Readiness Analysis

The analyzer also predicts **Cloud migration blockers**.

Typical blockers:

```
workflow scripts
unsupported plugins
large custom field counts
complex permission schemes
```

Outputs:

```
migration_risk_score
migration_effort_estimate
```

14. Enterprise Extensions

Advanced deployments add:

Machine learning anomaly detection

Detect unusual patterns:

```
issue spikes
workflow abuse
automation loops
```

Configuration drift detection

Compare instance snapshots over time.

---

15. Expected Diagnostic Coverage

A mature analyzer typically runs:

```
150–300 diagnostics
```

Covering:

```
schema
configuration
automation
performance
security
migration readiness
```

If you want, I can also generate the **next level of this system** that most Jira admins never build but becomes extremely powerful:

- **Automatic JQL performance profiler**
- **Workflow graph visualizer**
- **Automation dependency mapper**
- **Custom field index cost estimator**
- **Full Jira configuration knowledge graph**

This turns the analyzer into a **complete architecture intelligence platform for Jira**.